

So Many Attention Mechanisms — Which One Should You Actually Use?

DARTFormer, from Cambridge, Imperial, and Oxford, brings differentiable architecture search to attention selection — and tests whether mixing several attention types beats picking the best single one.

Ever since the Transformer arrived, researchers have produced a long menu of cheaper stand-ins for its self-attention: Longformer, Bigbird, Performer, Reformer, Linformer, Synthesizer, and more. The original dot-product attention scales quadratically with sequence length, and these efficient variants exist to cut that cost on long-sequence tasks.

The awkward part is choosing one. For a given long-range task, which efficient attention should you reach for? Earlier work on the Long Range Arena (LRA) benchmark delivered an uncomfortable answer: when models are trained from scratch, with no pretraining, the best attention depends heavily on the task, and none of them wins everywhere. That leaves practitioners with a brute-force fallback — train every attention and compare — which is absurdly expensive.

DARTFormer sets out to make attention selection both cheap and reliable, and along the way it takes on a question the field has left hanging: since different attention heads may specialize in different kinds of relationships, would a model that mixes several attention types beat any single one? The paper adapts differentiable architecture search (DARTS-style NAS) to the Transformer. It builds a supernet that runs every candidate attention in parallel, then ranks them with a cheap scoring signal.

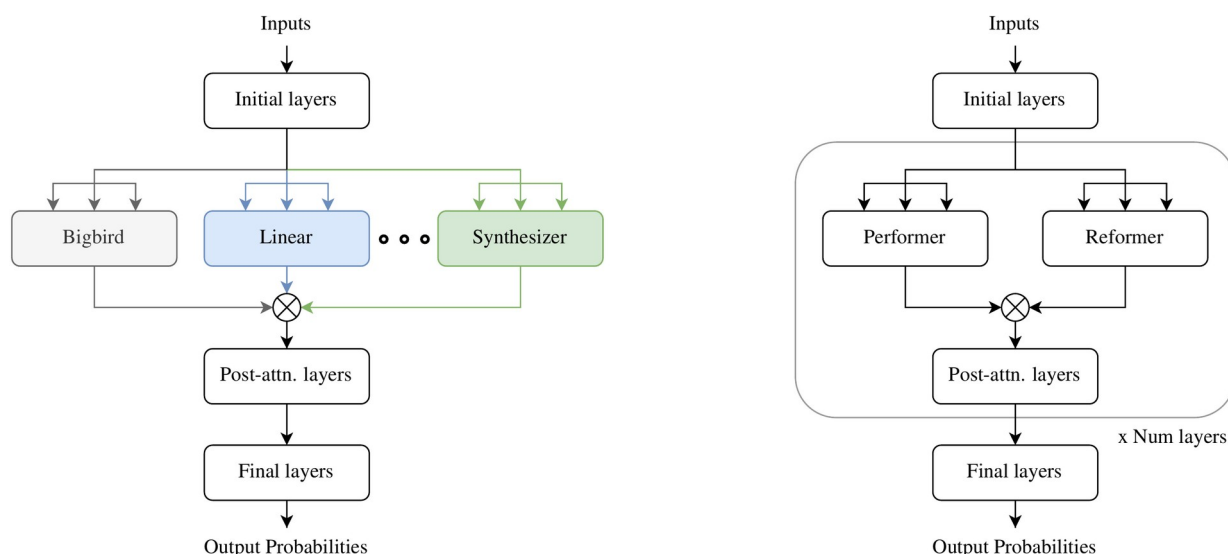


Figure 1. The DARTFormer supernet (left) and a derived final architecture (right). The supernet is a single-layer Transformer that runs many efficient attention blocks — Bigbird, Linear, Synthesizer, and others — in parallel and averages their outputs. The derived model on the right mixes two attention types, Performer and Reformer, inside one layer: heterogeneous across heads, homogeneous across layers.

Why picking an attention deserves its own method

In vision, convolutional kernels learn different features on their own, and there is a long-standing belief that a Transformer's attention heads similarly learn different relational biases. If that holds, a model that hosts several attention types at once ought to capture a richer set of relationships and outperform any single-attention model. Rather than assume this, DARTFormer treats it as a hypothesis to test and asks it directly: is the optimal attention for a task actually a mixture of several attentions?

Answering that by brute force is a non-starter — nine candidate mechanisms, plus their combinations, is far too much to train one at a time. That is the motivation for a search: evaluate every candidate in a single training run instead of training every candidate.

The core method: run every attention in parallel, then see who is missed most

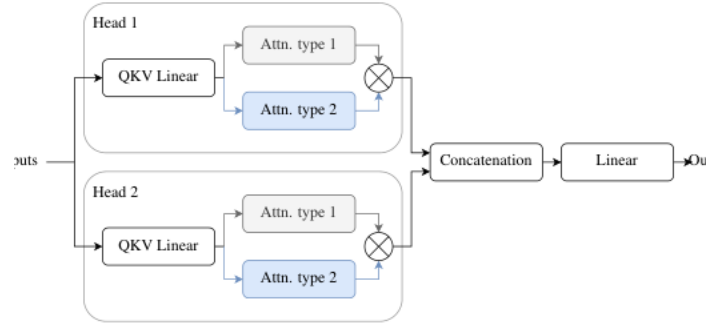
The first step is a single-layer supernet: each efficient attention becomes a candidate block, and all of them read the same input side by side. Here the paper borrows a simplification from DARTS — a fixed- α scheme. Instead of learning softmax edge weights or running an expensive bi-level optimization, it simply averages the candidate blocks' outputs: $\bar{o} = (1/N) \cdot \sum o_i$ over the N candidate blocks. The payoff is a supernet that trains all the way to convergence — stable and cheap.

Once it is trained, how do you score each attention? The paper uses a masked validation accuracy drop: mask out one attention block and measure how far validation accuracy falls. The larger the drop, the more that block mattered — the attention whose removal hurts most is judged best for the task. For a homogeneous model, you simply take the top-scoring block and drop it into a full Transformer.

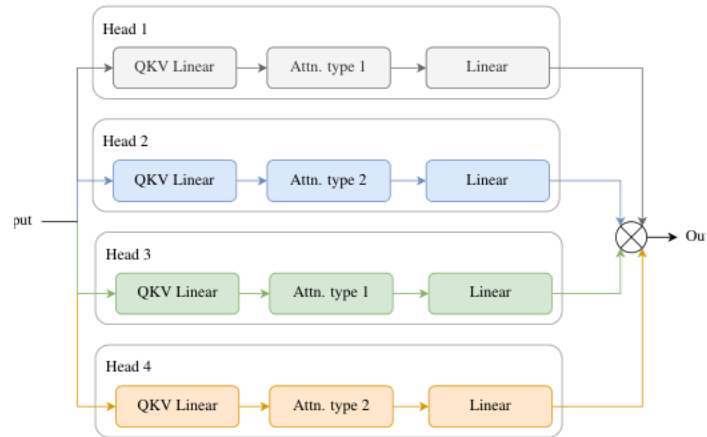
This cheap signal turns out to be surprisingly sharp. On LRA's ListOps task, Reformer earns a masked-validation-drop score of 11.85, while all eight other mechanisms score below 0.5 — a clean, almost unambiguous selection signal. On text classification the search likewise picks out Performer correctly. It is not infallible: on document matching, where the scores are small across the board, the single-layer supernet fails to place a true top-3 attention into its own top-3 picks.

Going further: can you search for a mixed-attention model?

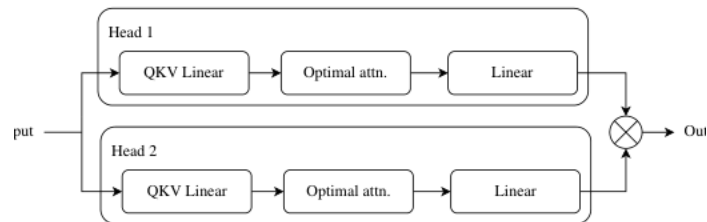
Since the supernet already holds several attentions together, the natural next thought is to keep more than one and assemble a heterogeneous Transformer. The paper pushes the search space down to the head level — each head is fixed to a single attention type, and the layer averages across heads. Figure 4 contrasts the theoretical version of this idea with the actual implementation, and shows the final form after the search, where every head is set to its optimal attention.



(a) Theoretical multi head attention supernet



(b) Actual multi head attention supernet



(c) Final multi attention layer after architecture search

Figure 4. The head-level search space. The theoretical version (a) lets each head choose an attention type internally; the actual implementation (b) fixes one attention type per head and averages across heads; (c) is the final multi-head layer after the search, with every head replaced by its optimal attention.

How do you pull a good mixture out of that space? The paper offers two routes. NAS Prune starts from a multi-head layer where each head carries a different attention, then repeatedly removes the lowest-scoring head and fine-tunes, until only the strongest heads remain (Figure 3). It is accurate but expensive. NAS One-shot skips the iteration entirely: it just takes the four top-scoring attentions from the single-layer selection and stacks them — far cheaper.

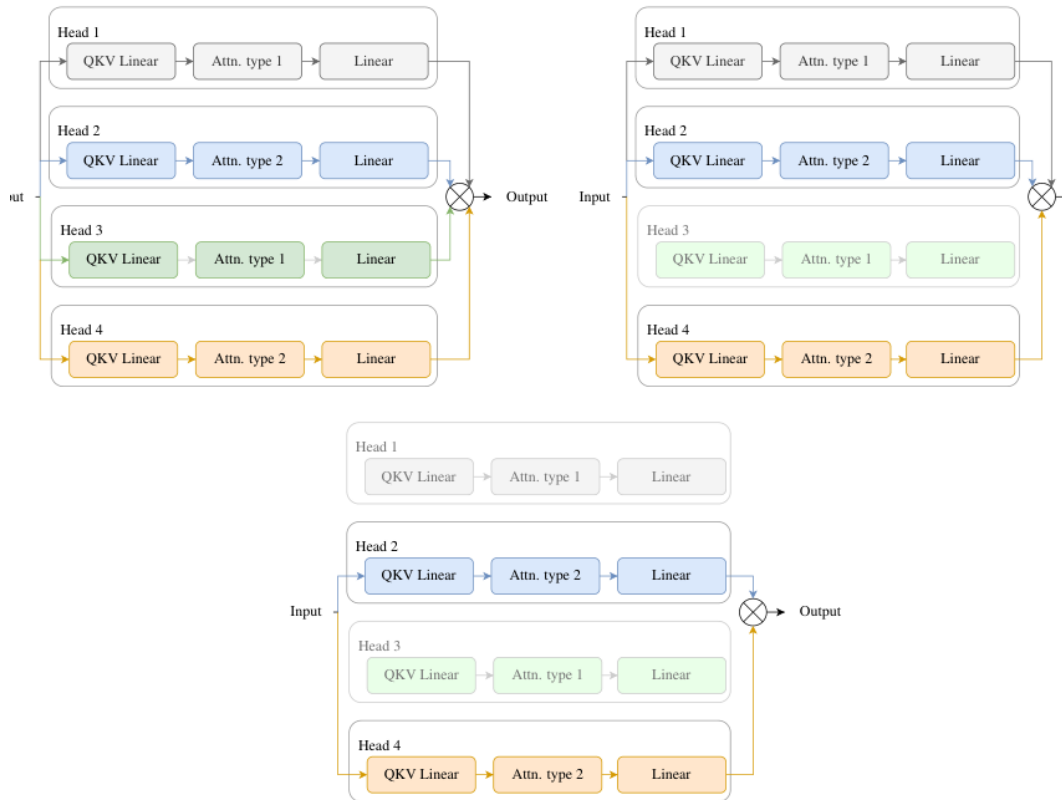


Figure 3. The NAS Prune procedure. Starting from a four-head layer where each head carries a different attention type, the method repeatedly removes the lowest-scoring head and fine-tunes, until only the strongest heads are left.

The key result: mixing beats the average, but not the best

The finding splits into two halves that point in opposite directions. The good news: as a way to pick the single best attention, the masked-validation-drop search genuinely works — it gets text classification and ListOps right. The bad news: once the authors assemble those attentions into a heterogeneous Transformer, neither the expensive NAS Prune nor the cheap NAS One-shot beats the best homogeneous model on any of the three tasks.

Table 1. Heterogeneous mixed-attention models vs. the best homogeneous model per task (full-model test accuracy, %). On all three tasks, both NAS Prune and NAS One-shot fall short of the best-homogeneous column.

Task	Best homogeneous	NAS Prune (heterog.)	NAS One-shot (heterog.)
Text classification	Performer 64.5	63.9	64.4
ListOps	Reformer 37.8	36.3	37.1
Document matching	Synthesizer 71.1	67.0	64.7

Numbers from the paper's Table 2. The gap is widest on document matching, where the heterogeneous models trail by roughly 4–6 points.

A useful practical aside comes from comparing the two search procedures: the expensive NAS Prune (iterative pruning plus fine-tuning) shows no consistent advantage over the cheap NAS One-shot (a single-pass top-four selection) — once the good attentions are correctly

identified, the cheap method is enough. Further attempts to tilt the mixture toward the strongest attentions, or give them more heads, produced no consistent gains either.

What to take away

For a new long-range task, instead of training all nine attentions, DARTFormer offers a cheap and reliable recipe: train a single-layer Transformer with the attentions run in parallel, and use masked validation accuracy drop to pick the winner. One caveat: when every candidate's drop score is small (as on document matching), the selection signal can be unreliable and deserves caution.

The more thought-provoking result is the counterintuitive one. Mixing several attention types does beat the average of picking one at random, but it never beats the best single choice. That sits at odds with the common assumption that diverse attention biases add up — at least on these long-range NLP tasks, choosing the right single attention beats blending many. The boundary of the claim is worth stating plainly: it rests on three LRA tasks, nine candidate attentions, and a from-scratch (no-pretraining) setup, and whether it survives other tasks or the addition of pretraining remains open.